

금융 규제 대응 감사 로그 SHA-256 체크섬과 불변성 보장

전자금융감독규정 §34 · 가상자산이용자보호법 §15 준수
AuditLogService — log() · verifyIntegrity() · queryByResource()

TypeScript

SHA-256

강의 20분 + 실습 35분

COINCRAFT
ACADEMY

왜 감사 로그가 필요한가?

금융 규제 3가지 + APPEND-ONLY 원칙

규정	요건	AuditLogService 구현
전자금융감독규정 §34	접근 기록 1년 이상 보존	INSERT + actor + 타임스탬프
가상자산이용자보호법 §15	거래 기록 5년 보존	S3 Glacier 백업 정책
ISMS-P	암호 무결성 검증	SHA-256 checksum 필드

✅ Append-only 원칙

INSERT만 허용

보정도 새 INSERT (이전 레코드 유지)

DB RLS(Row Level Security)로 강제

UPDATE/DELETE 정책 없음 = 묵시적 차단

❌ Append-only 위반 예시

UPDATE audit_log SET action = 'MODIFIED'

DELETE FROM audit_log WHERE id = 42

→ 위반 즉시 규정 위반 + checksum 불일치 감지됨

audit_log 테이블 + Row Level Security

POSTGRESQL — INSERT-ONLY 강제

```
CREATE TABLE audit_log (  
  id          BIGSERIAL PRIMARY KEY,  
  event_time  TIMESTAMPTZ NOT NULL,  
  actor       TEXT NOT NULL,      -- 행위자 (userId 또는 'system')  
  action      TEXT NOT NULL,      -- 'MINT_REQUESTED', 'KRW_MINT', ...  
  resource_type TEXT NOT NULL,    -- 'MintRequest', 'KRWToken', ...  
  resource_id TEXT NOT NULL,      -- 리소스 식별자  
  before_state JSONB,            -- 변경 전 상태 (null 가능)  
  after_state  JSONB NOT NULL,    -- 변경 후 상태  
  ip_address   TEXT,  
  session_id   TEXT,  
  checksum     CHAR(64) NOT NULL  -- SHA-256 hex (64자)  
);  
  
-- RLS: INSERT만 허용 (app_role 기준)  
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;  
CREATE POLICY audit_insert_only ON audit_log  
  FOR INSERT TO app_role WITH CHECK (true);  
-- UPDATE/DELETE 정책 없음 = DB 레벨에서 묵시적 차단
```

AuditEntry · LogParams · VerifyResult 타입

AUDITLOGSERVICE.TS — 인터페이스 3종

```
export interface LogParams {           // log() 호출 시 전달
  actor:      string;
  action:     string;
  resourceType: string;
  resourceId: string;
  beforeState?: unknown;  // 변경 전 상태 (optional)
  afterState:  unknown;   // 변경 후 상태 (필수)
  ipAddress?:  string;
  sessionId?:  string;
}

export interface VerifyResult {        // verifyIntegrity() 반환
  id:          number;
  valid:        boolean;
  storedChecksum: string;  // DB에 저장된 값
  computedChecksum: string; // 재계산한 값 - 같으면 valid: true
}
```

beforeState vs afterState — beforeState는 optional. 최초 생성 이벤트(MINT_REQUESTED)는 이전 상태가 없다. afterState는 필수 — "무엇이 됐는가"가 핵심 증거.

SHA-256 checksum 생성 원리

레코드별 독립 체크섬 — 5개 필드 연결 → 64자 hex

```
checksum = SHA-256(eventTime + actor + action + resourceId + JSON(afterState))
```

예시:

```
eventTime = "2026-05-18T10:00:00.000Z"
```

```
actor = "system"
```

```
action = "KRW_MINT"
```

```
resourceId = "0xminthash"
```

```
afterState = {"amount": "1000000", "status": "confirmed"}
```

```
raw = "2026-05-18T10:00:00.000ZsystemKRW_MINT0xminthash{\"amount\":\"1000000\",\"status\":\"confirmed\"}"
```

```
checksum = sha256(raw) → 64자 소문자 hex
```

핵심 규칙 — 5개 필드(eventTime · actor · action · resourceId · afterState)를 구분자 없이 이어붙여 SHA-256 해시. **beforeState**는 포함하지 않는다 — afterState만으로 "무엇이 됐는가"를 보호하면 충분하고, beforeState가 null인 경우 계산이 복잡해짐.

generateChecksum() — Node.js crypto 구현

AUDITLOGSERVICE.TS — createHash('sha256') 사용법

```
import { createHash } from 'crypto';

private generateChecksum(
  eventTime: Date, actor: string, action: string,
  resourceId: string, afterState: unknown,
): string {
  const raw = eventTime.toISOString() + actor + action
    + resourceId + JSON.stringify(afterState);
  return createHash('sha256').update(raw, 'utf8').digest('hex');
}
```

구분자 없음의 함정

"A"+"BC" 와 "AB"+"C" 는 raw가 동일 → 필드 경계 충돌 가능. 실운영에서는 이벤트 타입 조합이 고정돼 실용적으로 무해하나, 엄격하게는 구분자 추가 고려.

Date vs string 주의

파라미터는 Date 객체로 받아 내부에서 toISOString() 호출. string으로 받으면 포맷 불일치 → verifyIntegrity() 재계산 시 불일치 발생.

checksum 필드별 역할 — 무결성 보장 원리

5개 필드 중 하나라도 바뀌면 checksum 불일치 → 변조 즉시 감지

actor 수정

```
UPDATE audit_log  
SET actor = 'admin'  
WHERE id = 55
```

- raw 재계산 시 actor 달라짐
- storedChecksum ≠ computed
- **valid: false**

afterState 수정

```
UPDATE audit_log  
SET after_state = '{...}'  
WHERE id = 42
```

- JSON.stringify 결과 달라짐
- checksum 불일치
- **valid: false**

eventTime 위조

```
UPDATE audit_log  
SET event_time = '...'  
WHERE id = 33
```

- toISOString() 결과 달라짐
- raw 달라짐 → 불일치
- **valid: false**

레코드 삭제 시도

```
DELETE FROM audit_log WHERE id = 20  
→ RLS 정책이 DELETE 차단 (DB 레벨)  
→ 삭제 자체가 불가 (checksum 이전 방어)
```

3중 방어 요약

RLS — UPDATE/DELETE 차단 (DB 레벨)
checksum — 단건 내용 변조 감지
verifyIntegrity(id) — 언제든지 재검증 가능

DatabaseClient 인터페이스 — raw SQL 추상화

ORM 없이 직접 쿼리 — 감사 로그의 특수성

```
// AuditLogService 내부에서 사용하는 DB 클라이언트 인터페이스
interface DatabaseClient {
    query(sql: string, params?: unknown[]): Promise<{ rows: Record<string, unknown>[] }>;
}

// 생성자 — DatabaseClient 주입
export class AuditLogService {
    constructor(private readonly db: DatabaseClient) {}
}
```

왜 ORM을 쓰지 않나?

- ORM이 UPDATE 쿼리를 자동 생성할 수 있음
- Append-only 강제를 ORM 레벨에서 보장 어려움
- raw SQL로 의도 명확히 표현
- 감사 로그는 ORM 추상화가 오히려 위험

테스트용 In-Memory Mock

- 실제 PostgreSQL 없이 테스트 가능
- INSERT 쿼리 파싱 → rows[] 배열에 추가
- SELECT WHERE id = ? → rows 필터
- checksum 조작 테스트 가능

log() — 전체 구현

감사 로그 INSERT + checksum 자동 생성

```
async log(params: LogParams): Promise<number> {
  const eventTime = new Date();
  const checksum = this.generateChecksum(
    eventTime, params.actor, params.action, params.resourceId, params.afterState,
  );
  const { rows } = await this.db.query(
    `INSERT INTO audit_log
      (event_time, actor, action, resource_type, resource_id,
       before_state, after_state, ip_address, session_id, checksum)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10) RETURNING id`,
    [ eventTime.toISOString(),
      params.actor, params.action, params.resourceType, params.resourceId,
      params.beforeState !== undefined ? JSON.stringify(params.beforeState) : null,
      JSON.stringify(params.afterState),
      params.ipAddress ?? null, params.sessionId ?? null, checksum ],
  );
  return rows[0]['id'] as number;
}
```

verifyIntegrity() — 단일 레코드 무결성 검증

저장된 checksum과 재계산한 checksum 비교

```
async verifyIntegrity(id: number): Promise<VerifyResult> {
  const { rows } = await this.db.query(
    'SELECT * FROM audit_log WHERE id = $1', [id],
  );
  if (rows.length === 0) throw new Error(`AuditLog not found: ${id}`);

  const row = rows[0]!;
  const storedChecksum = row['checksum'] as string;
  const computedChecksum = this.generateChecksum(
    new Date(row['event_time'] as string),
    row['actor'] as string,
    row['action'] as string,
    row['resource_id'] as string,
    JSON.parse(row['after_state'] as string),
  );

  return { id, valid: storedChecksum === computedChecksum, storedChecksum, computedChecksum };
}
```

핵심 로직 — DB에서 읽은 값으로 checksum을 다시 계산해 저장된 값과 비교. 누군가 after_state나 actor를 수정했다면 재계산 결과가 달라져 **valid: false** 반환.

queryByResource() / queryByActor() — 조회 메서드

감사 추적 및 규정 보고용 쿼리

```
async queryByResource(
  resourceType: string, resourceId: string,
  opts?: { limit?: number; offset?: number },
): Promise<AuditEntry[]> {
  const limit = opts?.limit ?? 100;
  const offset = opts?.offset ?? 0;
  const { rows } = await this.db.query(
    `SELECT * FROM audit_log WHERE resource_type = $1 AND resource_id = $2 ORDER BY event_time ASC LIMIT $3 OFFSET $4`,
    [resourceType, resourceId, limit, offset],
  );
  return rows.map(this._rowToEntry);
}

async queryByActor(actor: string, from: Date, to: Date): Promise<AuditEntry[]> {
  const { rows } = await this.db.query(
    `SELECT * FROM audit_log WHERE actor = $1 AND event_time BETWEEN $2 AND $3 ORDER BY event_time ASC`,
    [actor, from.toISOString(), to.toISOString()],
  );
  return rows.map(this._rowToEntry);
}
```

_rowToEntry() — DB 행 → AuditEntry 변환

JSON 역직렬화 + 타입 캐스팅

```
private _rowToEntry(row: Record<string, unknown>): AuditEntry {
  return {
    id:          row['id']          as number,
    eventTime:   new Date(row['event_time'] as string),
    actor:       row['actor']       as string,
    action:      row['action']      as string,
    resourceType: row['resource_type'] as string,
    resourceId:  row['resource_id']  as string,
    beforeState: row['before_state']
      ? JSON.parse(row['before_state'] as string)
      : undefined,
    afterState:  JSON.parse(row['after_state'] as string),
    ipAddress:  row['ip_address'] as string | undefined,
    sessionId:  row['session_id'] as string | undefined,
    checksum:   row['checksum']   as string,
  };
}
```

beforeState 처리 — null이면 undefined 반환. JSON.parse(null)는 에러. row['before_state'] truthy 체크 필수. afterState는 항상 존재하므로 null 체크 불필요.

prevChecksum Race Condition — 동시성 함정

다중 Worker 환경에서 체인 방식이 깨지는 이유 + 해결책

문제 — 두 Worker가 동시에 log() 호출

Worker A: `SELECT checksum ... ORDER BY id DESC LIMIT 1`
→ `prevChecksum = 'abc123'` ← 현재 마지막

Worker B: `SELECT checksum ... ORDER BY id DESC LIMIT 1`
→ `prevChecksum = 'abc123'` ← 동일 값 조회!

Worker A: `generateChecksum('abc123' + ...)` → `'def456'`
`INSERT` → `id=101`

Worker B: `generateChecksum('abc123' + ...)` → `'ghi789'`
`INSERT` → `id=102` ← `id=100` 기준 계산됨!

`verifyChainIntegrity()`: `id=102`에서 체인 불일치 ●

해결책 1 — SELECT FOR UPDATE (트랜잭션)

```
await db.transaction(async trx => {
  const last = await trx.query(
    `SELECT checksum FROM audit_log
     ORDER BY id DESC LIMIT 1
     FOR UPDATE` ← 행 잠금
  );
  // Worker B는 A의 COMMIT까지 대기
  const prev = last.rows[0]?.checksum ?? '';
  // ... INSERT
});
```

해결책 2 — PQueue 단일 Writer

```
private readonly queue =
  new PQueue({ concurrency: 1 });

async log(params) {
  return this.queue.add(async () => {
    // 직렬 실행 보장
    // 단, 다중 Pod 환경에서는 효과 없음
  });
}
```

감사 로그 훼손 시나리오 3가지

어떻게 감지되는가?

시나리오 1 · checksum 조작

```
UPDATE audit_log SET after_state = '{...}'  
WHERE id = 42
```

- verifyIntegrity(42) 호출
- generateChecksum() 재계산
- storedChecksum ≠ computed
- **valid: false**

시나리오 2 · actor 수정

```
UPDATE audit_log SET actor = 'admin'  
WHERE id = 55
```

- raw에 'admin' 포함 시
- checksum이 달라짐
- verifyIntegrity(55)
- **valid: false**

시나리오 3 · 가짜 레코드 삽입

id=100-101 사이에 가짜 INSERT

- id=101의 prevChecksum은
원래 id=100의 checksum을 가리킴
- 가짜 레코드가 끼어들면
id=101 checksum 불일치
- **체인 끊김 감지**

RLS + checksum + 체인 — 3중 방어 — RLS: UPDATE/DELETE 차단 / checksum: 단건 내용 변조 감지 / 체인(prevChecksum): 삭제·삽입 순서 조작 감지. 계층별 역할이 다르다.

5년 보관 전략 — S3 Glacier 아카이빙

가상자산이용자보호법 §15 — 거래 기록 5년 보존

보관 계층 설계

- 1 PostgreSQL audit_log — 실시간 조회 (1년)
- 2 30일마다 S3 Export
- 3 S3 Standard — 90일 이후 자동 전환
- 4 S3 Glacier Instant Retrieval — 1825일(5년) 후 Expiration. 필요 시 수 시간 내 복원

S3 Lifecycle Policy 핵심

```
{
  "Rules": [{
    "Transitions": [{
      "Days": 90,
      "StorageClass": "GLACIER_IR"
    }],
    "Expiration": { "Days": 1825 }
  }]
}
```

보관 단계	기간	조회 속도	비용
PostgreSQL	0~1년	즉시	높음
S3 Standard	1~3개월	즉시	중간
S3 Glacier IR	3개월~5년	수초~수분	낮음

makeDb() In-Memory Mock — 테스트 구조

실제 PostgreSQL 없이 테스트 가능한 이유 · SQL 문자열 파싱 패턴

```
function makeDb() {
  let nextId = 1;
  const rows: Record<string, unknown>[] = [];
  return {
    rows,
    async query(sql, params = []) {
      if (sql.includes('INSERT INTO AUDIT_LOG')) {
        const [eventTime, actor, action,
              resourceType, resourceId, /*...*/ checksum] = params;
        const row = { id: nextId++, event_time: eventTime,
                      actor, action, resource_type: resourceType,
                      resource_id: resourceId, checksum, /* ... */ };
        rows.push(row);
        return { rows: [{ id: row.id }] };
      }
      if (sql.includes('WHERE id ='))
        return { rows: rows.filter(r => r.id === params[0]) };
      // resource_type, actor 필터 생략
    },
  };
}
```

테스트 구조 (9개 → 실제 11개):

그룹	케이스 수
log() 기본	6개 (id반환, DB저장, actor/action, checksum 64hex, beforeState 직렬화, beforeState null)
verifyIntegrity()	3개 (valid:true, 조작→false, 없는id→throw)
queryByResource()	2개 (beforeState 역직렬화, 해당 resource만)
queryByActor()	2개 (actor+시간 범위, 없으면 빈배열)

핵심 패턴:

checksum 조작 테스트: `row['checksum'] = '0'.repeat(64)`

→ `verifyIntegrity()`에서 `generateChecksum()` 재계산 → 불일치 → `valid: false`

커리큘럼 vs 실제 코드 — 체크섬 방식 비교

day07.md: SHA-256 체인(chained) · 실제 코드: 단일 레코드 독립 체크섬

커리큘럼 접근법 (체인)

```
checksum_n = SHA256(  
    prev_checksum + ← 이전 레코드 체크섬  
    eventTime + actor +  
    action + resourceId + afterState  
)
```

장점: 중간 레코드 1개 삭제 시
이후 전체 체크섬 불일치 → 감지
단점: 모든 레코드 순서대로 검증 필요
대규모 감사 시 성능 저하

실제 코드 (단일 레코드)

```
checksum = SHA256(  
    eventTime + actor + action +  
    resourceId + JSON(afterState)  
)  
// prev_checksum 없음
```

장점: 각 레코드 독립 검증 가능
`verifyIntegrity(id)` 한 번만 호출
병렬 검증 가능
단점: 레코드 삭제 자체는 체크섬으로
감지 불가 (RLS로 보완)

실제 코드가 단일 레코드를 선택한 이유:

RLS(Row Level Security)가 DELETE 자체를 차단하므로 체인이 불필요.

단일 레코드가 `verifyIntegrity(id)` 인자만으로 바로 검증 가능 → 운영 편의성 우선.

체인 방식은 Phase 3에서 감사 추적 강도 요건이 높아질 때 재검토 예정.

queryByActor() — 규정 감사 대응 시나리오

금감원 조사 요청 시 특정 담당자의 행위 기록 추출 · 날짜 범위 조회

실제 운영 시나리오:

금융감독원 서면 조사: "2026년 3월 1일 ~ 3월 31일 사이

운영자 kim@kyobo.com의 모든 행위 내역을 제출하라"

```
const logs =
  await auditLog.queryByActor(
    'kim@kyobo.com',
    new Date('2026-03-01T00:00:00.000Z'),
    new Date('2026-03-31T23:59:59.999Z'),
  );

// 각 레코드 무결성 개별 검증:
for (const entry of logs) {
  const result =
    await auditLog.verifyIntegrity(entry.id);
  if (!result.valid) {
    // 변조 감지 → 보고서에 플래그
  }
}
```

인덱스 설계:

```
-- queryByResource 최적화
CREATE INDEX idx_audit_resource
  ON audit_log(resource_type, resource_id);

-- queryByActor 최적화
CREATE INDEX idx_audit_actor_time
  ON audit_log(actor, event_time);

-- 감독원 조회는 대용량 → 인덱스 필수
-- event_time만 인덱스 걸면 actor 필터에 비효율
```

limit/offset 기본값:

queryByResource: limit=100, offset=0

queryByActor: 기본 limit 없음 → 날짜 범위로 제한

대용량 조회 시 pagination 필수

generateChecksum() 상세 — createHash + beforeState 처리

Node.js crypto 모듈 · beforeState를 checksum에 포함하지 않는 이유

```
import { createHash } from 'crypto';

private generateChecksum(
  eventTime: Date,
  actor: string,
  action: string,
  resourceId: string,
  afterState: unknown,
): string {
  // 5개 필드를 문자열로 이어 붙임
  const raw =
    `${eventTime.toISOString()}` // ISO 8601
    + actor // 구분자 없음
    + action
    + resourceId
    + JSON.stringify(afterState); // JSON

  return createHash('sha256')
    .update(raw, 'utf8')
    .digest('hex');
  // → 64자 소문자 hex 문자열
}
```

beforeState를 checksum에 포함 안 하는 이유:

beforeState는 "원래 어땠나"를 참조하는 정보.

checksum의 핵심은 "이 레코드에서 일어난 일"을 보호하는 것.

afterState만 포함해도 무결성 보장 충분.

+ beforeState가 null이면 checksum 계산에 포함하기 복잡해짐.

구분자 없음의 함정:

raw = "A" + "B" + "CD" 와 "AB" + "C" + "D" 는 동일

→ 필드 경계가 불명확하면 충돌 가능

실제 코드에서는 이벤트 타입 조합이 고정돼 실용적으로 무해.

엄격하게 하려면 구분자 추가 고려.

S26 완료 기준 — 전체 체크리스트

코드 구현 + 개념 설명 + 11개 테스트 통과 + M4 마무리

구현 · 코드

- `log()` — `eventTime+checksum` 생성 + `INSERT` + `id` 반환
- `verifyIntegrity(id)` — 재계산 체크섬 비교 + `VerifyResult`
- `queryByResource()` — `resource_type+resource_id` 필터
- `queryByActor()` — `actor` + 날짜 범위 필터
- `_rowToEntry()` — `beforeState` null 처리 + `JSON.parse`
- `generateChecksum()` — 5개 필드 raw 문자열 + `createHash('sha256')`
- 11개 테스트 전부 PASS

개념 · 설명

- 단일 레코드 vs 체인 체크섬 차이 설명 가능
- `beforeState`를 `checksum`에 포함하지 않는 이유
- RLS + `checksum` 이중 방어 원리
- `verifyIntegrity()`가 `id`를 받는 이유 (체인 아님)
- 규정 3가지: 전자금융 §34, 가상자산법 §15, ISMS-P
- S3 Glacier 5년 보관 전략과 비용 최적화

M4 완료 → M5 진입:

S27 `WalletProvisioningService` (EXTERNAL/KYOBO)
S28 `WalletMappingService` + auto-provisioning
S29 `EIP-191 verifyOwnership()` · S30 `EventConditionService`